

09

Routing & Navigation

© Gelişim · Junior → Mid-Level

📌 Bu modülde ne öğreneceğiz?

- ✓ Rotaların nasıl tanımlandığını, eşleştğini ve nasıl gezinildiğini
- ✓ URL'deki parametreleri `input()` ile doğrudan component'e almayı
- ✓ Guard'larla gezinmeyi durdurmayı, resolver'larla veriyi önden hazırlamayı
- ✓ Rotaları tembel (lazy) yükleyerek ilk açılışı hızlandırmayı
- ✓ İç içe rotalarla sabit bir kabuk (sidebar) kurmayı

[9.1 Temel Routing](#)[9.2 Route Parametreleri & Input Binding](#)[9.3 Functional Guards](#)[9.4](#)[Functional Resolvers](#)[9.5 Lazy Loading Rotalar](#)[9.6 İç İçe Rotalar](#)[Mini Proje: Rol Tabanlı](#)[Yönetim Paneli](#)[Özet](#)

Temel Routing

Modül 8'in sonunda uygulaman dış dünyayla konuşuyordu: veriyi çekiyor, yazıyor, hatayı yönetiyordu. Ama hepsi **tek bir ekranda** oluyordu. Bu modülde uygulamana **sayfa** kavramını kazandırıyoruz.

⇒ Önceki Modüllerden Hatırla

- ⇒ Modül 5: Durumu `signal()` içinde tutmak. Routing, durumun bir kısmını signal yerine **URL'de** tutmanın yoludur.
- ⇒ Modül 7: `@Service()` + `inject()`. Guard ve resolver'lar da `inject()` kullanır.
- ⇒ Modül 8: `httpResource` ile veri çekmek. Bu modülde veriyi **hangi ürünün** çekileceğini URL belirleyecek.

Nedir?

Router, adres çubuğundaki URL'e bakıp hangi component'i göstereceğine karar eden yapıdır. Sunucuya yeni bir sayfa istemez; aynı sayfa içinde ekranı değiştirir, ama URL'i de günceller.

Neden var? "Signal ile ekran değiştirsem olmaz mı?"

Teknik olarak olur: bir `currentPage = signal('list')` tutup şablonda `@switch` ile ekran değiştirebilirsin. Ama üç şeyi kaybedersin:

Kaybettiğin şey	Neden önemli
Paylaşılabilir adres	Kullanıcı bir ürünün linkini arkadaşına gönderemez; herkes hep ana ekranda açar.
Geri / ileri tuşu	Tarayıcının geri tuşu uygulamandan tamamen çıkar — kullanıcı için kırık his.
Yer imi ve yenileme	<code>F5</code> 'e basınca bulunduğun yerde kalman gerekir; signal'de tutulan ekran sıfırlanır.

Kural şu: **kullanıcının geri tuşuyla dönmek isteyeceği ya da linkini paylaşmak isteyeceği her durum URL'e aittir.** Gerisi signal'de kalır.

Nasıl? Üç parça

Routing üç parçadan oluşur ve üçü de olmadan çalışmaz:

1) Rota tablosu — hangi yol hangi component:

TYPESCRIPT

```
// app.routes.ts
import { Routes } from '@angular/router';
import { ProductList } from '../pages/product-list/product-list';
import { ProductDetail } from '../pages/product-detail/product-detail';
```

```
import { NotFound } from './pages/not-found/not-found';

export const routes: Routes = [
  { path: '', redirectTo: 'products', pathMatch: 'full' },
  { path: 'products', component: ProductList, title: 'Ürünler' },
  { path: 'products/:id', component: ProductDetail },
  { path: '**', component: NotFound },          // en SONDA olmalı
];
```

2) Router'ı sağlamak — uygulama yapılandırmasında:

TYPESCRIPT

```
// app.config.ts

import { provideRouter, withComponentInputBinding } from '@angular/router';

export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes, withComponentInputBinding()),
    provideHttpClient(withInterceptors([baseUrlInterceptor])),
  ],
};
```

`withComponentInputBinding()` şu an gereksiz görünebilir; 9.2'de bunun sayesinde URL parametrelerinin doğrudan `input()` 'lara aktığını göreceğiz. Baştan ekle.

3) Çıkış noktası ve bağlantılar — şablonda:

TYPESCRIPT

```
@Component({
  selector: 'app-root',
  imports: [RouterOutlet, RouterLink, RouterLinkActive],
  template: `
    <nav>
      <a routerLink="/products" routerLinkActive="active">Ürünler</a>
      <a routerLink="/about" routerLinkActive="active">Hakkında</a>
    </nav>

    <!-- Eşleşen component TAM BURAYA basılır -->
    <router-outlet />
  `,
})
```

```
})  
export class App {}
```

⚠ routerLink mi href mi?

`<a href="/products">` tarayıcıya “**bu adresi sunucudan yeniden iste**” der: sayfa baştan yüklenir, uygulamanın bütün durumu sıfırlanır, kullanıcı beyaz ekran görür. `routerLink` ise Router'a haber verir; ekran değişir ama sayfa yenilenmez.

Yol yazımları

Yazım	Anlamı
<code>path: ''</code>	Kök adres. Genelde bir <code>redirectTo</code> ile eşleşir.
<code>path: 'products'</code>	Sabit yol: <code>/products</code>
<code>path: 'products/:id'</code>	Parametrelili yol: <code>/products/42</code> → <code>id = '42'</code>
<code>path: '**'</code>	Hiçbiri tutmazsa (404). Dizinin en sonunda olmalı.
<code>redirectTo: 'products'</code>	Yönlendirme. Boş yolda <code>pathMatch: 'full'</code> ile birlikte.
<code>title: 'Ürünler'</code>	Sekme başlığı. Router bunu otomatik <code>document.title</code> 'a yazar.

⚠ pathMatch: 'full' neden şart?

Yol eşleştirme varsayılan olarak “**prefix**” modundadır: `path: ''` her URL'in başında bulunduğu için **her adresle eşleşir**. Yönlendirmen sonsuz döngüye girer ya da hiçbir sayfaya ulaşamazsın. `'full'` ise “URL'in tamamı boş olsun” der.
Dolu yollarda (`'products'`) gerekmez.

Koddan gezinme

Şablondan `routerLink`, koddan `Router`. Kaydetme bittikten sonra listeye dönmek gibi durumlarda ikincisi gerekir:

TYPESCRIPT

```
import { inject } from '@angular/core';  
import { Router } from '@angular/router';  
  
export class ProductCreate {  
  private readonly router = inject(Router);  
  private readonly productService = inject(ProductService);  
  
  save(): void {
```

```
this.productService.add(/* ... */).subscribe(() => {  
  // Dizi biçimi: her eleman bir yol parçası  
  this.router.navigate(['/products']);  
});  
}  
}
```

Dizi biçimini tercih et: `['/products', id]` parçaları senin yerine güvenli biçimde birleştirir. Metin birleştirmede (`'/products/' + id`) özel karakterler seni yakar.

🔔 Mini Quiz

1. Routing üç parçadan oluşur — hangileri?

↳ Rota tablosu (`routes`), `provideRouter()` ve `<router-outlet />` (+ `routerLink`).

2. `pathMatch: 'full'` neden yalnızca boş yolda gerekir?

↳ Eşleştirme varsayılan olarak “prefix”tir; boş yol o modda **her** URL ile eşleşir.

3. `***` rotası neden dizinin sonunda olmalı?

↳ Router yukarıdan aşağıya **ilk eşleşeni** seçer; `***` her şeyle eşleştiği için kendinden sonrakileri gölgeler.

4. Şablondan ve koddan gezinme sırasıyla neyle yapılır?

↳ `routerLink` ve `inject(Router).navigate(...)`.

✗ Sık Yapılan Hatalar

✗ **href kullanmak.** Tam sayfa yeniler, uygulama durumu sıfırlanır. Uygulama içi her bağlantı `routerLink`.

✗ **`routerLink="/products/{{ id }}"` yazmak.** Bu metin enterpolasyondur ve karışıklık üretir; parametrelili gezinmede `[routerLink]="['/products', id]"` kullan.

✗ **`<router-outlet />` 'i unutmak.** Rota eşleşir, hata da almazsın — ekranda hiçbir şey görünmez.

✗ **`***` 'ı listenin başına koymak.** Bütün uygulaman 404 sayfası olur.

✓ Bu Dersten Çıkarılacaklar

✓ Router = URL → component eşleştiricisi; SPA'ya geri tuşu ve paylaşılabılır adres kazandırır.

✓ Üç parça: `routes` + `provideRouter()` + `<router-outlet />`.

✓ Kullanıcının linkini paylaşacağı ya da geri tuşuyla döneceği durum URL'e, gerisi signal'e ait.

✓ Boş yolda `pathMatch: 'full'`, `***` en sonda.

✓ Koddan gezinme `router.navigate(...)` — dizi biçimiyle.

Ders 9.2

<> Route Parametreleri & Component Input Binding

Rota tablosunu kurdun. Şimdi asıl soru: `/products/42` adresindeki **42** component'in içine nasıl girecek?

Nedir? URL'in dört bilgi kaynağı

Router bir rotayı aktive ederken elinde dört ayrı bilgi kümesi olur:

Kaynak	Örnek	Ne zaman kullanılır
Yol parametresi (path param)	<code>/products/42</code> → <code>id</code>	Kaynağı tanımlayan bilgi. Onsuz sayfa anlamsız.
Sorgu parametresi (query param)	<code>?q=kahve&page=2</code>	Sayfayı ayarlayan bilgi: arama, sayfa, sıralama, filtre.
Statik veri	<code>data: { role: 'admin' }</code>	Rotaya sabitlenmiş yapılandırma.
Resolver verisi	<code>resolve: { product }</code>	Rota açılmadan önce çözülmüş veri (9.4).

Neden ikisi ayrı? (Sık karıştırılır)

Ölçüt basit: **o bilgi olmadan sayfa var olabiliyor mu?** Ürün detayı `id` olmadan var olamaz → yol parametresi. Ürün listesi arama terimi olmadan gayet iyi çalışır, terim sadece görünümü daraltır → sorgu parametresi.

Nasıl? `input()` ile doğrudan bağlama

`provideRouter(routes, withComponentInputBinding())` eklendiğinde Router, bu dört kaynaktaki değerleri **aynı ada sahip** `input()` 'lara kendiliğinden yazar. Ek bir abonelik ya da kurulum kodu yazmazsın:

```

// Rota: { path: 'products/:id', component: ProductDetail }

@Component({ /* ... */ })
export class ProductDetail {
  // Ada dikkat: rotadaki ':id' ile birebir aynı olmalı
  readonly id = input.required<string>();

  private readonly productService = inject(ProductService);

  // id() değişince URL değişir, httpResource kendini yeniler (Modül 8)
}
```

```
readonly product = httpResource<Product>(() => `products/${this.id()}`);  
}
```

⚠ Yol parametreleri her zaman metindir

`input.required<number>()` yazsan bile URL'den gelen değer `'42'` (string) olur. TypeScript bunu derlerken yakalayamaz, çünkü değeri Router çalışma anında yazar. Sonuç: `id === 42` hep `false`, `id + 1` ise `'421'`.

Çözüm sayı gerekiyorsa dönüştürücü kullanmak:

TYPESCRIPT

```
import { input, numberAttribute } from '@angular/core';  
  
readonly id = input.required({ transform: numberAttribute }); // number  
readonly page = input(1, { transform: numberAttribute }); // varsayılan 1
```

Sorgu parametreleri

Aynı mekanizma sorgu parametreleri için de çalışır:

TYPESCRIPT

```
// URL: /products?q=kahve&page=2  
  
export class ProductList {  
  readonly q = input(''); // arama terimi  
  readonly page = input(1, { transform: numberAttribute }); // sayfa  
}
```

Şablondan sorgu parametresiyle gezinmek:

HTML

```
<!-- Sadece sayfayı değiştir, aramayı KORU -->  
<a [routerLink]="['/products']"  
  [queryParams]="{ page: page() + 1 }"  
  queryParamsHandling="merge">Sonraki</a>
```

queryParamsHandling	Davranış
'merge'	Yeni parametreleri mevcutlarla birleştirir. Sayfalama/filtre için doğru seçim.
'preserve'	Mevcutları aynen korur, yenileri yok sayar.

<code>queryParamsHandling</code>	Davranış
<code>'replace'</code>	Varsayılan. Mevcutları siler, yalnızca yenileri kalır.

Varsayılanın `'replace'` olduğunu unutma: sayfa değiştiren bir bağlantıya `queryParamsHandling` yazmazsan kullanıcının arama terimi sessizce kaybolur.

Mutfak detayı: aynı rotada parametre değişirse ne olur?

`/products/1` sayfasındayken `/products/2` 'ye gidersen Router component'i **yeniden oluşturmaz** — aynı component örneğini kullanır, yalnızca `input` değerini günceller. Bu bilinçli bir tasarım (performans), ama iki sonucu var:

Yaklaşım	Ne olur?
<code>ngOnInit</code> içinde veriyi çekmek	Bir daha çalışmaz. Ekranda hâlâ 1 numaralı ürün durur.
<code>httpResource(() => `products/\${id()}`)</code>	Kendiliğinden yenilenir. <code>id()</code> bir signal olduğu için.

Modül 5'ten beri kurduğumuz signal alışkanlığı burada bedavaya kazandırıyor: `input()` zaten bir signal, `httpResource` onu okuduğu için bağımlılık kurulmuş oluyor.

❏ Eşleşmeyen `input` ne olur?

Bir `input` 'a karşılık gelen anahtar URL'de yoksa Router ona `undefined` yazar — eski değer **kalmaz**. Bu, kullanıcı `?q=kahve` 'yi silince ekranda eski aramanın asılı kalmasını engeller. Bu yüzden sorgu parametresi bağlayan input'lara **varsayılan değer ver**: `input('')` , `input(1, { transform: numberAttribute })` .

🕒 Çakışma olursa kim kazanır?

Aynı ada sahip birden çok kaynak varsa öncelik sırası şudur (zayıftan güçlüye): **sorgu parametresi** → **yol parametresi** → **statik `data`** → **resolver verisi**. Yani resolver her zaman son sözü söyler.

🔔 Mini Quiz

- Bir bilginin yol mu sorgu parametresi mi olduğuna nasıl karar verirsin?**
↳ “O bilgi olmadan sayfa var olabilir mi?” Olamıyorsa yol, olabiliyorsa sorgu parametresi.
- `id = input.required<number>()` yazdın ama karşılaştırmalar tutmuyor. Neden?
↳ URL'den gelen değer **her zaman string'dir**; tip yalnızca derleme zamanı iddiasıdır. `transform: numberAttribute` kullan.
- Sayfa numarasını değiştiren bağlantıda arama terimi kayboluyor. Eksik olan ne?
↳ `queryParamsHandling="merge"` . Varsayılan `'replace'` diğer parametreleri siler.
- `/products/1` 'den `/products/2` 'ye geçince veri neden tazelenmedi?

↳ Veriyi `ngOnInit` 'te çektin. Component yeniden oluşmaz; `httpResource` gibi signal okuyan bir kaynak kullan.

× Sık Yapılan Hatalar

- × **URL parametresini sayı sanmak.** Hep string gelir; `numberAttribute` ile dönüştür.
- × `input` adını rotadaki parametreden farklı yazmak. Hata almazsın, değer sessizce `undefined` kalır.
- × `withComponentInputBinding()` 'i eklemeyi unutmak. Aynı sessiz `undefined` : rota çalışır, parametre gelmez.
- × **Filtreyi yalnızca signal'de tutmak.** Kullanıcı filtrelediği listenin linkini paylaşamaz, `F5` her şeyi sıfırlar.

✓ Bu Dersten Çıkarılacaklar

- ✓ Yol parametresi kaynağı **tanımlar**, sorgu parametresi görünümü **ayarlar**.
- ✓ `withComponentInputBinding()` + aynı adlı `input()` = ek kod olmadan bağlama.
- ✓ Gelen değer her zaman string; sayı için `transform: numberAttribute`.
- ✓ Aynı rotada parametre değişince component yeniden oluşmaz — signal tabanlı kaynak (`httpResource`) kullan.
- ✓ `queryParamsHandling="merge"` olmadan diğer parametreler silinir.

Ders 9.3

🌟 Functional Guards — Gezinmeyi Denetlemek

Rotalar artık çalışıyor. Ama `/admin` adresini adres çubuğuna yazan herkes oraya girebiliyor. Guard'lar bu kapıyı tutan bekçilerdir.

Nedir?

Guard, bir gezinme başladığında çalışan ve “bu gezinme devam etsin mi?” sorusunu yanıtlayan bir fonksiyondur. Sıradan bir fonksiyondur; içinde `inject()` kullanabilirsin.

Dört tür guard

Tür	Ne zaman çalışır	Tipik iş
<code>canActivate</code>	Rota aktive edilmeden hemen önce	Giriş yapılmış mı, rolü yeterli mi
<code>canActivateChild</code>	Çocuk rotalar aktive edilmeden önce	Bir bölümün tamamını tek yerden korumak
<code>canMatch</code>	Rota eşleştirilirken	Rotanın hiç eşleşmemesini sağlamak (aşağıda)

Tür	Ne zaman çalışır	Tipik iş
canDeactivate	Rotadan çıkarken	“Kaydedilmemiş değişiklik var” uyarısı

Nasıl? En yalın guard

```
// guards/admin-guard.ts

import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
import { AuthService } from '@app/services/auth-service';

export const adminGuard: CanActivateFn = () => {
  const auth = inject(AuthService);
  const router = inject(Router);

  if (auth.role() === 'admin') {
    return true;
  }

  // false yerine YÖNLENDİRME döndür – kullanıcı neden giremediğini görsün
  return router.createUrlTree(['/unauthorized']);
};
```

```
// app.routes.ts

{ path: 'admin/new-product', component: ProductCreate, canActivate: [adminGuard] },
```

Guard ne döndürebilir?

Dönüş	Sonuç
true	Gezinme devam eder.
false	Gezinme iptal edilir . Kullanıcı bulunduğu sayfada kalır.
UrlTree	Gezinme iptal edilir ve bu adrese yönlendirilir .
RedirectCommand	Yönlendirme + ek seçenek (ör. geçmişte iz bırakmadan).
Observable / Promise	Yukarıdakilerin asenkron hâli; Router bekler.

❓ false mi, UrlTree mi?

`false` döndürmek kullanıcıyı olduğu yerde bırakır: tıklar, hiçbir şey olmaz, nedenini anlamaz. Bir sayfa yetki istiyorsa `router.createUrlTree(['/unauthorized'])` ile açıklayıcı bir ekrana gönder. `false` 'ı asıl `canDeactivate` 'te (kullanıcı “Hayır, kalmak istiyorum” dediğinde) kullanırsın.

`canActivate` ile `canMatch` farkı

İkisi de erişimi engeller ama **farklı anda** çalışır ve bu fark önemlidir:

	<code>canActivate</code>	<code>canMatch</code>
Ne zaman	Rota eşleşti, aktive edilecek	Rota eşleştirilirken
Engellerse	Gezinme durur / yönlenir	Rota hiç eşleşmemiş sayılır , arama devam eder
Lazy chunk	İndirilmiş olur	İndirilmez
Kullanımı	“Girebilir mi?”	“Bu rota bu kullanıcı için var mı?”

`canMatch` rota eşleştirmesini etkilediği için şuna izin verir: **aynı adres, role göre farklı component.**

TYPESCRIPT

```
export const routes: Routes = [  
  // Router yukarıdan aşağıya bakar. adminGuard geçerse ilki eşleşir...  
  { path: 'reports', component: AdminReports, canMatch: [adminGuard] },  
  // ...geçmezse ilk rota hiç yokmuş gibi davranılır ve bu eşleşir  
  { path: 'reports', component: BasicReports },  
];
```

`canDeactivate` — çıkışı tutmak

Diğerlerinden farklı olarak bu guard **component örneğini** alır; böylece bileşenin durumuna bakabilir:

TYPESCRIPT

```
// guards/unsaved-changes-guard.ts  
export interface HasUnsavedChanges {  
  hasUnsavedChanges(): boolean;  
}  
  
export const unsavedChangesGuard: CanDeactivateFn<HasUnsavedChanges> = (component) => {  
  if (!component.hasUnsavedChanges()) {  
    return true;  
  }  
}
```

```
return confirm('Kaydedilmemiş değişiklikler var. Yine de çıkmak istiyor musun?');
};
```

⚠ Guard içinde `inject()` neden çalışıyor?

Guard'lar sıradan fonksiyonlardır; `inject()` ise yalnızca “injection context” denen anlarda çalışır. Router, guard'ı bilinçli olarak bu bağlamda çağırır — o yüzden **fonksiyonun ilk satırlarında** `inject()` serbesttir.

Ama `setTimeout` ya da bir `subscribe` geri çağırımının **içinde** `inject()` çağırırsan bağlam çoktan kapanmıştır ve hata alırsın. Kuralı basit tut: **bütün `inject()` çağrılarını en üstte yap**, sonra mantığı yaz.

🔔 Mini Quiz

1. Guard `false` döndürdüğünde kullanıcı nerede kalır?
↳ Bulunduğu sayfada — hiçbir şey olmamış gibi. Nedenini göstermek için `createUrlTree(...)` ile yönlendir.
2. Lazy yüklenen bir yönetici bölümünü korurken hangi guard'ı seçersin, neden?
↳ `canMatch` : rota eşleşmediği için lazy chunk hiç indirilmez.
3. `canDeactivate` 'i diğerlerinden ayıran nedir?
↳ Component örneğini parametre olarak alır; bileşenin durumuna bakabilir.
4. Guard yetkiyi denetliyorsa API'de ayrıca kontrol gerekir mi?
↳ Kesinlikle evet. Guard bir arayüz kolaylığıdır; tarayıcıda çalışan her kod değiştirilebilir. Yetki denetiminin gerçeği sunucudadır.

✗ Sık Yapılan Hatalar

- ✗ **Guard'ı güvenlik sanmak.** Guard yalnızca arayüzü düzenler; gerçek denetim sunucuda yapılır.
- ✗ **Guard içinde veri çekmek.** Guard “girebilir mi?” sorusunu yanıtlar; veri hazırlamak resolver'ın işidir (9.4).
- ✗ **`inject()` 'i geri çağırımın içinde kullanmak.** Injection context kapanmıştır; çağrılar fonksiyonun en üstünde yap.
- ✗ **Yalnızca `false` döndürmek.** Kullanıcı tıklar, hiçbir şey olmaz, nedenini anlamaz.

✓ Bu Dersten Çıkarılacaklar

- ✓ Guard = gezinmeyi denetleyen sıradan fonksiyon; içinde `inject()` çalışır.
- ✓ `canActivate` girişi, `canMatch` eşleşmeyi, `canDeactivate` çıkışı denetler.
- ✓ `UrlTree` döndürmek `false` döndürmekten neredeyse her zaman iyidir.
- ✓ `canMatch` ile aynı adres role göre farklı component'e gidebilir.

✓ Guard arayüz kolaylığıdır — yetkinin gerçeği sunucuda denetlenir.

Ders 9.4

🚀 Functional Resolvers — Veriyi Önden Hazırlamak

Guard “girebilir mi?” diye sorar. Resolver ise “**girmeden önce şu veriyi hazırla**” der.

Nedir?

Resolver, rota aktive edilmeden önce çalışan ve döndürdüğü veri hazır olana kadar gezinmeyi bekleten bir fonksiyondur. Component açıldığında veri çoktan elindedir.

Neden? İki farklı açılış hissi

Resolver'sız	Resolver'lı
Sayfa hemen açılır, içerik boştur; iskelet döner, sonra veri gelir.	Kullanıcı bir an bulunduğu sayfada bekler, sonra dolu sayfaya geçer.
Ürün yoksa: sayfa açılır, sonra “bulunamadı” gösterilir.	Ürün yoksa: sayfa hiç açılmaz , listeye geri yönlendirilir.

İkisi de doğrudur; seçim veriye bağlıdır. Küçük ve hızlı bir kayıt için resolver daha derli toplu bir his verir. Büyük listelerde ise resolver uygulamayı **donmuş gibi** gösterir — orada iskelet daha iyidir.

Nasıl?

TYPESCRIPT

```
// resolvers/product-resolver.ts
import { inject } from '@angular/core';
import { ResolveFn, RedirectCommand, Router } from '@angular/router';
import { catchError, of } from 'rxjs';

export const productResolver: ResolveFn<Product | RedirectCommand> = (route) => {
  const productService = inject(ProductService);
  const router = inject(Router);

  const id = Number(route.paramMap.get('id'));

  return productService.getById(id).pipe(
    // Ürün yoksa sayfayı hiç açma, listeye gönder
    catchError(() => of(new RedirectCommand(router.parseUrl('/products')))),
  );
}
```

```
);  
};
```

TYPESCRIPT

```
// app.routes.ts  
{  
  path: 'products/:id',  
  component: ProductDetail,  
  resolve: { product: productResolver }, // anahtar adı: 'product'  
},
```

Component tarafı 9.2'deki kalıbın aynısı — resolver'daki **anahtar adı**yla aynı adı taşıyan bir `input` yeter:

TYPESCRIPT

```
export class ProductDetail {  
  // resolve: { product: ... } -> input adı da 'product'  
  readonly product = input.required<Product>();  
}
```

Dikkat: burada `httpResource`, `isLoading`, `hasValue()` kontrolü yok. Veri garanti hazır olduğu için component sadeleşir. Resolver'ın asıl kazancı budur.

⚠ Mutfak detayı: resolver ne zaman yeniden çalışır?

Varsayılan davranış `paramsChange` 'dir: resolver yalnızca **yol parametresi** değişince yeniden çalışır. Sorgu parametresi değişince çalışmaz.

Yani `?page=2` 'den `?page=3` 'e geçtiğinde resolver'ın verisi **eski kalır** ve bunu fark etmek zordur: hata yok, ekran değişmiyor. Gerekliyse rotaya `runGuardsAndResolvers: 'paramsOrQueryParamsChange'` ekle.

💡 Mini Quiz

1. Resolver ile guard arasındaki temel fark nedir?

↳ Guard gezinmeye izin verir/vermez; resolver gezinmeden önce veri hazırlar.

2. Resolver kullanınca component'te hangi kodlar gereksizleşir?

↳ Yükleniyor durumu ve `hasValue()` korumaları — veri garanti hazırdır.

3. Sayfa numarası sorgu parametresiyle değişiyor ama resolver verisi tazelenmiyor. Neden?

↳ Varsayılan `paramsChange` yalnızca yol parametresini izler; `runGuardsAndResolvers: 'paramsOrQueryParamsChange'` gerekir.

× Sık Yapılan Hatalar

- × **Her veriyi resolver'a taşımak.** Yavaş isteklerde uygulama donmuş görünür; büyük listeler için `httpResource` + iskelet daha iyidir.
- × **Resolver'da hatayı ele almamak.** İstek patlarsa gezinme sessizce iptal olur ve kullanıcı hiçbir şey olmamış sanır.
- × **Resolver anahtarı ile `input` adını farklı yazmak.** Yine sessiz `undefined`.

✓ Bu Dersten Çıkarılacaklar

- ✓ Resolver = rota açılmadan veriyi hazırlayan fonksiyon; component sadeleşir.
- ✓ Anahtar adı (`resolve: { product }`) ile `input` adı aynı olmalı.
- ✓ `RedirectCommand` ile “kayıt yoksa hiç açma, yönlendir” davranışı kurulur.
- ✓ Varsayılanda yalnızca yol parametresi değişince yeniden çalışır.
- ✓ Her yerde değil: yavaş/büyük veride iskelet göstermek daha iyi bir histir.

Ders 9.5

🎯 Lazy Loading — Rotaları Tembel Yüklemek

Uygulaman büyüdükçe ilk açılıшта indirilen JavaScript de büyür. Kullanıcı belki hiç girmeyeceği yönetici ekranlarının kodunu da indirir. Lazy loading bunu düzeltir.

Nedir? Neden?

Lazy loading, bir rotanın kodunu ancak o rotaya gidildiğinde indirmektir. Kazanç ilk açılış süresinde görülür: kullanıcı gördüğü ekranın kodunu indirir, gerisini indirmez.

Nasıl? İki biçim

Tek component: `loadComponent`

TYPESCRIPT

```
{
  path: 'reports',
  loadComponent: () => import('./pages/reports/reports').then((m) => m.Reports),
}
```

Bir rota grubu: `loadChildren`

```
// app.routes.ts
{
  path: 'admin',
  loadChildren: () => import('./features/admin/admin.routes').then((m) => m.adminRoutes),
}

// features/admin/admin.routes.ts
export const adminRoutes: Routes = [
  { path: '', component: AdminHome },
  { path: 'users', component: AdminUsers },
];
```

⚠ Sık yapılan yanlış: üstte `import` edip `loadComponent` yazmak

Şunu yazarsan lazy loading **olmaz**:

```
import { Reports } from './pages/reports/reports'; // <-- bu satır her şeyi bozar

{ path: 'reports', loadComponent: () => Promise.resolve(Reports) }
```

Dosyanın en üstündeki `import` satırı, o component'i zaten ana pakete katmıştır. Ayırmayı sağlayan şey **dinamik** `import()` çağrısıdır. Kural: üstte `import` ettiğin component için `component:` kullan; ayırmak istiyorsan **yalnızca** dinamik `import()` yaz.

Guard ile birleşince

9.3'teki fark burada işe yarıyor: `canActivate` chunk indikten sonra çalışır, `canMatch` ise indirmeden önce. Yetkisiz kullanıcıya yönetici kodunu hiç göndermek istemiyorsan `canMatch` kullan:

```
{
  path: 'admin',
  canMatch: [adminGuard], // önce bak
  loadChildren: () => import('./features/admin/admin.routes')
    .then((m) => m.adminRoutes), // sonra indir
}
```

Önden yükleme (preloading)

Lazy loading'in bedeli, kullanıcının tıkladığı anda kısa bir bekleme olmasıdır. Preloading, uygulama açıldıktan **sonra**, boşta kalan zamanda lazy paketleri arka planda indirir:

```
import { provideRouter, withPreloading, PreloadAllModules } from '@angular/router';

provideRouter(routes, withComponentInputBinding(), withPreloading(PreloadAllModules)),
```

Böylece ilk açılış hızlı kalır *ve* gezinme anında olur. Yalnızca `canMatch` ile korunan rotalar bu durumda da indirilmez.

⇒ Bunu Nasıl Doğrularsın?

- ⇒ Modül 3: DevTools **Network** sekmesi. Lazy loading'i gerçekten doğrulamanın tek yolu: rotaya tıkladığında yeni bir `.js` dosyasının indiğini görmek.

💡 Mini Quiz

1. Lazy loading'i sağlayan şey tam olarak nedir?
 - ↳ Dinamik `import()` çağrısı. Üstte statik `import` varsa ayırma gerçekleşmez.
2. Yetkisiz kullanıcıya yönetici kodunu hiç göndermemek için hangi guard?
 - ↳ `canMatch` — rota eşleşmediği için chunk indirilmez.
3. Preloading lazy loading'in hangi dezavantajını giderir?
 - ↳ Tıklama anındaki kısa bekleme; paketler boşta kalan zamanda önden indirilir.

✓ Bu Dersten Çıkarılacaklar

- ✓ Lazy loading = kodu ancak gerekince indirmek; kazanç ilk açılıştadır.
- ✓ `loadComponent` tek component, `loadChildren` rota grubu içindir.
- ✓ Ayırmayı **dinamik** `import()` sağlar; statik import her şeyi bozar.
- ✓ `canMatch` + lazy = yetkisiz kullanıcı kodu hiç indirmez.
- ✓ `withPreloading(PreloadAllModules)` ile ikisinin de iyi yanını al.

Ders 9.6

🚀 İç İç Rotalar — Sabit Kabuk, Değişen İçerik

Yönetim panellerinin hepsi aynı iskelete sahiptir: solda sabit bir menü, sağda değişen içerik. Menüyü her sayfaya kopyalamak yerine **iç iç rotalar** kullanırsın.

Nedir?

Bir rotanın `children` dizisi varsa, o rotanın component'i bir “kabuk” olur ve çocuk rotalar onun içindeki ikinci bir `<router-outlet />` 'e basılır.

Nasıl?

TYPESCRIPT

```
export const routes: Routes = [
  { path: '', redirectTo: 'panel', pathMatch: 'full' },
  {
    path: 'panel',
    component: Dashboard,           // KABUK: sidebar + <router-outlet />
    children: [
      { path: '', redirectTo: 'overview', pathMatch: 'full' },
      { path: 'overview', component: Overview },
      { path: 'products', component: ProductList },
      { path: 'products/:id', component: ProductDetail },
    ],
  },
];
```

TYPESCRIPT

```
@Component({
  selector: 'app-dashboard',
  imports: [RouterOutlet, RouterLink, RouterLinkActive],
  template: `
    <div class="shell">
      <aside class="sidebar">
        <!-- Başında / YOK: görelî yol -> /panel/products -->
        <a routerLink="products" routerLinkActive="active">Ürünler</a>
        <a routerLink="overview" routerLinkActive="active">Genel Bakış</a>
      </aside>

      <section class="main">
        <router-outlet />  <!-- çocuk rota BURAYA -->
      </section>
    </div>
  `,
})
export class Dashboard {}
```

```
/panel/products    -> Dashboard > ProductList
/panel/products/42 -> Dashboard > ProductDetail
/panel/overview    -> Dashboard > Overview
```

Dashboard yeniden oluşturulmaz; yalnızca içteki outlet değişir.

⚠ Başındaki / her şeyi değiştir

`routerLink="products"` **görelidir**: bulunduğu rotanın altına gider → `/panel/products` .
`routerLink="/products"` ise **mutlaktır**: kökten başlar → `/products` . Kabuğun içindeyken bunu yazarsan ya 404 alırsın ya da paneli terk edersin.

Görelî gezinme

Yazım	Nereye gider
<code>routerLink="products"</code>	Bir alta: <code>/panel/products</code>
<code>routerLink=".."</code>	Bir üste: detaydan listeye
<code>routerLink="../.."</code>	İki üste
<code>router.navigate(['..'], { relativeTo: route })</code>	Koddan görelî gezinme — <code>relativeTo</code> olmadan mutlak sayılır

🔖 `routerLinkActive` alt rotalarda da açık kalır

`routerLink="products"` taşıyan menü bağlantısı, `/panel/products/42` adresindeyken de **aktif** sayılır — çünkü eşleşme varsayılan olarak “bu yolla başlıyor mu?” diye bakar. Genelde istediğin budur (mini projede de öyle).

Yalnızca tam eşleşmede aktif olsun istiyorsan: `[routerLinkActiveOptions]="{ exact: true }"` .

Rota kapsamında servis

Modül 7’de `@Service({ autoProvided: false })` ile bir servisi component kapsamına almıştın. Aynıı rota düzeyinde de yapılabilir: `providers` yazılan rotada ve tüm çocuklarında **tek bir örnek** yaşar, o rotadan çıkılınca yok olur.

TYPESCRIPT

```
{
  path: 'panel',
  component: Dashboard,
  providers: [DashboardFilterService], // yalnızca panel içinde yaşar
  children: [ /* ... */ ],
}
```

🔔 Mini Quiz

1. Bir rotayı “kabuk” yapan nedir?

↳ `children` dizisi ve component'inde ikinci bir `<router-outlet />`.

2. `routerLink="products"` ile `routerLink="/products"` farkı?

↳ İlki görelidir (bulunduğun rotanın altına), ikincisi mutlak (kökten).

3. Kabuğun çocuk rotaları neden ikinci bir `<router-outlet />` ister?

↳ Birinci outlet kabuğu basar; çocuklar kabuğun içinde bir yere basılmalıdır.

✗ Sık Yapılan Hatalar

- ✗ **Çocuk bağlantılarda başa / koymak.** Göreli sanırsın, mutlak olur; panelden çıkarsın.
- ✗ **Kabuk component'inde `<router-outlet />` unutmak.** Menü görünür, içerik hiç gelmez — hata da yoktur.
- ✗ **Boş çocuk rotada `pathMatch: 'full'` yazmamak.** Yönlendirme her çocuk adresle eşleşir ve döngüye girer.

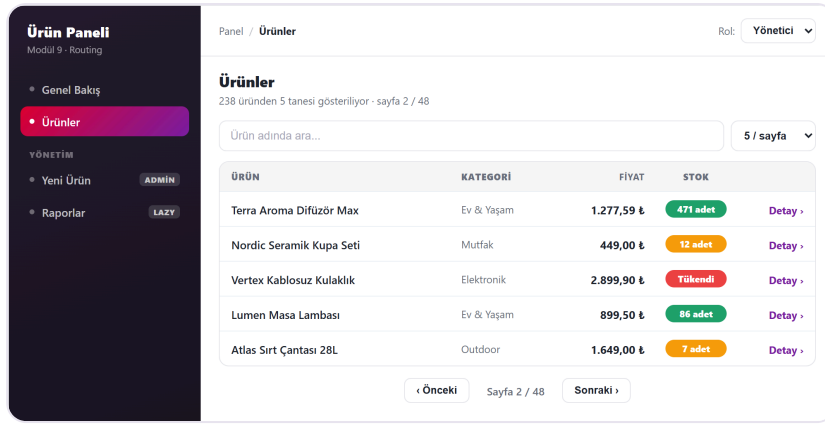
✓ Bu Dersten Çıkarılacaklar

- ✓ `children` + ikinci `<router-outlet />` = sabit kabuk, değişen içerik.
- ✓ Kabuk yeniden oluşturulmaz; menü durumu ve kaydırma korunur.
- ✓ Başında `/` olmayan `routerLink` görelidir.
- ✓ `routerLinkActive` alt rotalarda da aktiftir; `{ exact: true }` ile daraltılır.
- ✓ Rota düzeyinde `providers` ile servis o bölüme kapsanır.

Bölüm

🚀 Mini Proje: Rol Tabanlı Yönetim Paneli

Bu modülün altı dersini tek bir uygulamada birleştiriyoruz: sabit sidebar'lı bir panel, URL'de yaşayan arama ve sayfalama, rota parametresiyle açılan ürün detayı, role göre kapanan sayfalar ve tembel yüklenen bir raporlar bölümü.



Ana ekran: kabuk (sidebar) + ürün listesi. Arama ve sayfa numarası URL'de sorgu parametresi olarak geçer.

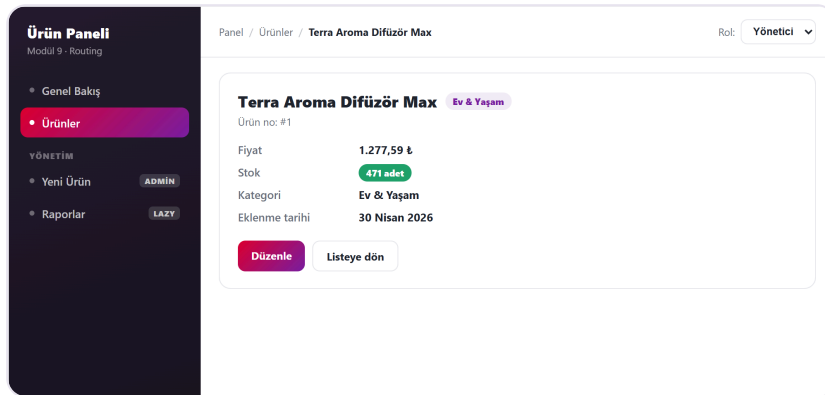
Rota haritası

TYPESCRIPT

```

'' ..... -> redirect 'panel'
'panel' ..... -> Dashboard (KABUK: sidebar + <router-outlet />)
'' ..... -> redirect 'genel-bakis' (pathMatch: 'full')
'genel-bakis' .. -> Overview
'urunler' ..... -> ProductList (sorgu param: q, page, pageSize)
'urunler/:id' .. -> ProductDetail (yol param + resolver)
'yeni-urun' .... -> ProductCreate (canActivate: adminGuard,
                                canDeactivate: unsavedChangesGuard)
'raporlar' ..... -> LAZY loadComponent + canMatch: adminGuard
'yetkisiz' ..... -> Denied
'**' ..... -> NotFound

```

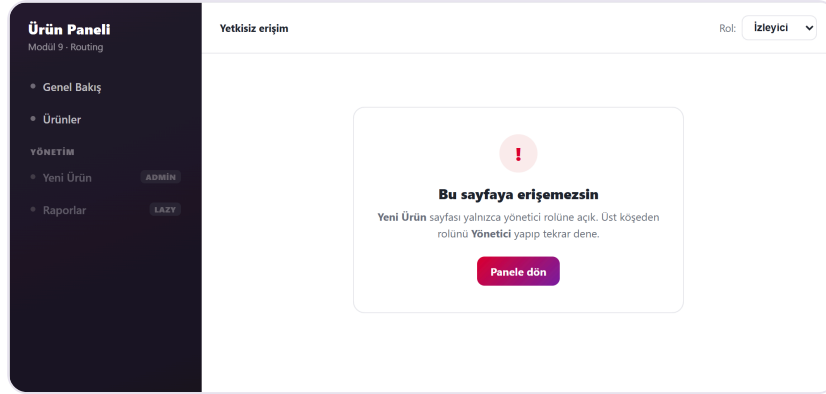


Ürün detayı: id yol parametresinden gelir, ürün resolver ile hazır gelir — yükleniyor ekranı yok.

Özellikler / Gereksinimler

- **Kabuk:** sol menü ve üst şerit sabit kalsın; yalnızca sağdaki alan değişsin (iç içe rotalar).
- **Liste:** ürünler API'den sayfalı gelsin. Arama terimi, sayfa ve sayfa boyutu **URL'de sorgu parametresi** olarak dursun — adresi kopyalayıp yeni sekmede açınca aynı ekran gelmeli.

- **Detay:** “Detay” bağlantısı `/panel/urunler/:id` adresine gitsin; ürün **resolver** ile hazır gelsin. Ürün yoksa sayfa hiç açılmasın, listeye yönlensin.
- **Rol:** üst şeritteki seçici rolü `'admin'` / `'viewer'` arasında değiştirsin (bir servisteki signal).
- **Yeni Ürün:** yalnızca admin girebilsin (`canActivate`). Viewer denerse `/yetkisiz` ekranına yönlensin.
- **Kaydedilmemiş değişiklik:** form doluyken sayfadan çıkılmak istenirse onay sorulsun (`canDeactivate`).
- **Raporlar:** tembel yüklensin ve `canMatch` ile korunsun — viewer rolündeyken chunk hiç inmemeli (Network sekmesinden doğrula).
- **404:** tanımsız adresler için bir sayfa olsun.
- **Biçimlendirme:** fiyat `currency` pipe ile, stok rozeti kendi directive'inle renklensin (Modül 6).



Viewer rolündeyken yönetici sayfası denendiğinde: guard düz false değil, yönlendirme döndürür.

Angular tarafı (sende) — ipuçları

- **Başlangıç:** `provideRouter(routes, withComponentInputBinding())` — bunu ilk iş olarak ekle, yoksa parametreler sessizce `undefined` gelir.
- **Rol servisi:** `@Service()` içinde `private readonly _role = signal<'admin' | 'viewer'>('admin')`, dışarı `asReadonly()` + bir `setRole()` metodu (Modül 7 kalıbı).
- **Liste:** `q`, `page`, `pageSize` birer `input()` olsun; `httpResource` bunları okuyunca URL değişince veri kendiliğinden tazelenir. Sayısal olanlarda `transform: numberAttribute` unutma.
- **Arama kutusu:** değeri doğrudan servise yazmak yerine `router.navigate([], { queryParams: { q, page: 1 }, queryParamsHandling: 'merge' })` ile URL'e yaz — tek doğruluk kaynağı URL olsun.
- **Sayfalama:** aynı mantık; `page` 'i `merge` ile güncelle, arama kaybolmasın.
- **Detay bağlantısı:** kabuk içindesin — görelî yaz: `[routerLink]="[product.id]"`.
- **Resolver:** `ResolveFn<Product | RedirectCommand>`; hata durumunda `new RedirectCommand(router.parseUri('/panel/urunler'))`.
- **Guard'lar:** `inject()` çağrılarını fonksiyonun en üstünde yap. `adminGuard` hem `canActivate` hem `canMatch` olarak kullanılabilir — ikisinde de aynı fonksiyon.
- **Kilitli menü:** viewer rolünde menüdeki yönetici bağlantılarına `[class.locked]="!isAdmin()"` ver — guard'ı UI'da da göster.
- **Lazy:** raporlar için **yalnızca** dinamik `import()`; dosyanın üstüne o component'i import etme.

⇒ Bu Projede Birleşen Modüller

- ⇒ Modül 5: `signal` / `computed` , immutable güncelleme.
- ⇒ Modül 6: `currency` pipe ve stok rozeti için custom directive (`host` + selector adıyla aynı input).
- ⇒ Modül 7: `@Service()` + `inject()` , servisin dışarı sade bir yüzey vermesi.
- ⇒ Modül 8: `httpResource` , `HttpClient` ile yazma, interceptor'lar, `?delay=1500` ile yükleniyor ekranını denemek.
- ⇒ Modül 9: rotalar, parametreler, guard'lar, resolver, lazy loading, iç içe rotalar.

📄 Materyaller ve API

Statik referansı ayrıca verdim (JS yok): `index.html` , `detail.html` , `denied.html` , `styles.scss` , `styles.css` ve üç ekran görüntüsü. HTML'lerin başındaki yorum blokları rota haritasını ve her parçanın Angular karşılığını anlatıyor.

API olarak Modül 8'deki `Module08ProductApi` 'yi kullan: `/api/products` (sorgu parametreleri: `name` , `page` , `pageSize` , `sort`) ve `/api/products/{id}` . Resolver'ın yönlendirme dalını denemek için var olmayan bir id ile `/panel/urunler/999999` adresini aç.

✓ Bu Dersten Çıkarılacaklar

- ✓ Ekran durumunun bir kısmı URL'e taşındı: adres artık paylaşılabılır.
- ✓ Kabuk + çocuk rota deseni: menü bir kez yazılır, içerik değişir.
- ✓ Guard'lar arayüzü düzenler; `canMatch` ayrıca kod indirmeyi de engeller.
- ✓ Resolver, detay sayfasındaki yükleniyor/boş durum kodunu tamamen kaldırdı.
- ✓ Önceki beş modülün araçları (signal, pipe, directive, servis, HTTP) burada tek bir uygulamada bulundu.

Bölüm

🚩 Modül 9 Özeti

Uygulaman artık tek ekranlı değil: adresi olan, geri tuşu çalışan, linki paylaşılabilen bir uygulama.

- **Kurulum:** `routes` + `provideRouter(routes, withComponentInputBinding())` + `<router-outlet />` .
- **Gezinme:** şablondan `routerLink` , koddan `router.navigate(...)` .
- **Parametreler:** aynı adlı `input()` ile bağlanır; değerler string gelir, sayı için `numberAttribute` .
- **Guard'lar:** `canActivate` / `canMatch` / `canDeactivate` — `false` yerine `UrlTree` .
- **Resolver:** veri hazır gelsin, component sadeleşsin; her yerde değil.
- **Lazy:** dinamik `import()` ; `canMatch` ile birleşince chunk hiç inmez.
- **İç içe rotalar:** `children` + ikinci outlet = sabit kabuk.

❗ Sırada ne var?

Modül 10 — Project Structure & Feature-Based Architecture: Dokuz modüldür dosyaları biraz da sezgiyle yerleştiriyoruz. Sırada bunun kuralları var: `core` / `shared` / `features` ayrımı, bağımlılık yönü kuralları, barrel export'lar, `tsconfig.json` yol takma adları ve mimariyi ESLint ile korumak. Bu modülde kurduğun rota haritası, oradaki klasör yapısının iskeleti olacak.

Takıldığın her yeri sor — mini projeyi yapıp gösterdiğinde birlikte gözden geçiririz. Kolay gelsin!